Python Libraries for NW

Tushar B. Kute,
http://tusharkute.com

Agenda

- Part 1: datetime - Mastering time manipulation, timezones, and duration.
- Part 2: hashlib - Ensuring data integrity and implementing secure password storage.
- Part 3: shodan - Leveraging Open Source Intelligence (OSINT) and network reconnaissance.

Handling Time in Python

- Built-in standard library module.
- Essential for logging, scheduling, and processing timestamps.
- Abstract complex calendar and time math away from the developer.

Naive vs. Aware Datetimes

Naive

Aware

Naive vs. Aware Datetimes

- Naive:
  - Contains no timezone info. Assumes local system time (e.g., "3:00 PM").
- Aware:
  - Explicitly includes timezone info (e.g., "3:00 PM in Tokyo").
- Best Practice:
  - Always use Aware datetimes for web apps, databases, and APIs.

The Foundation of datetime

- datetime.date: Represents Year, Month, Day.
- datetime.time: Represents Hour, Minute, Second, Microsecond.
- datetime.datetime: Combines both Date and Time.

```
today = datetime.date.today()
now = datetime.datetime.now()
```

Handling Timezones (Python 3.9+)

- Historically required third-party libraries (like pytz).
- Python 3.9 introduced the built-in zoneinfo module.
- Uses the OS's IANA Time Zone Database.

```
from zoneinfo import ZoneInfo
tokyo_tz = ZoneInfo("Asia/Tokyo")
now_tokyo = datetime.datetime.now(tokyo_tz)
```

Calculating Durations with timedelta

Calculating Durations with timedelta

- Represents the difference between two dates/times.
- Can be added or subtracted from datetime objects.

```
two_weeks = datetime.timedelta(days=14)
future_date = now + two_weeks
```

Formatting and Parsing

- strftime (String Format Time): Object → String.
- strptime (String Parse Time): String → Object.

```
# Format
now.strftime("%Y-%m-%d %H:%M:%S")
# Parse
datetime.datetime.strptime("2024-10-25", "%Y-%m-%d")
```

What is hashlib?

- Python's built-in module for secure hashes and message digests.
- Hash Function: Converts arbitrary data (text, files) into a fixed-size string of characters.
- Operates on bytes, not strings.

What Makes a Secure Hash?

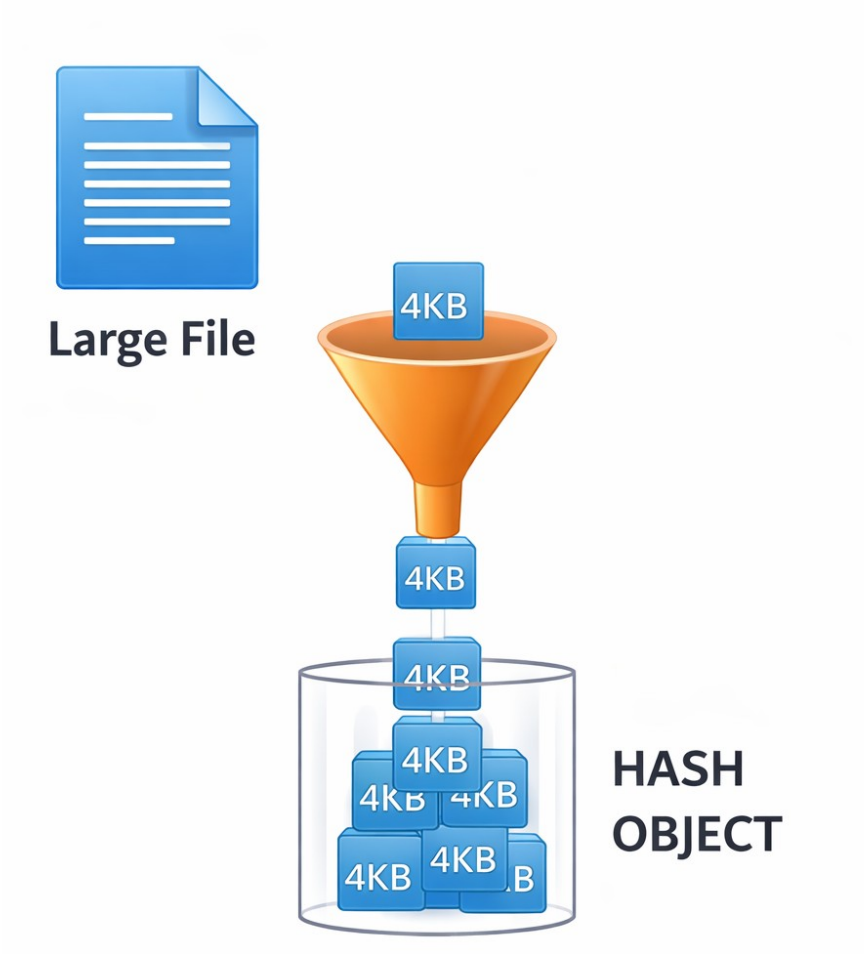
- Deterministic: Same input = exact same output.
- One-Way: Impossible to reverse-engineer the original data.
- Avalanche Effect: Changing 1 bit completely changes the hash.
- Collision Resistant: Hard to find two inputs with the same hash.

Hashing Data with SHA-256

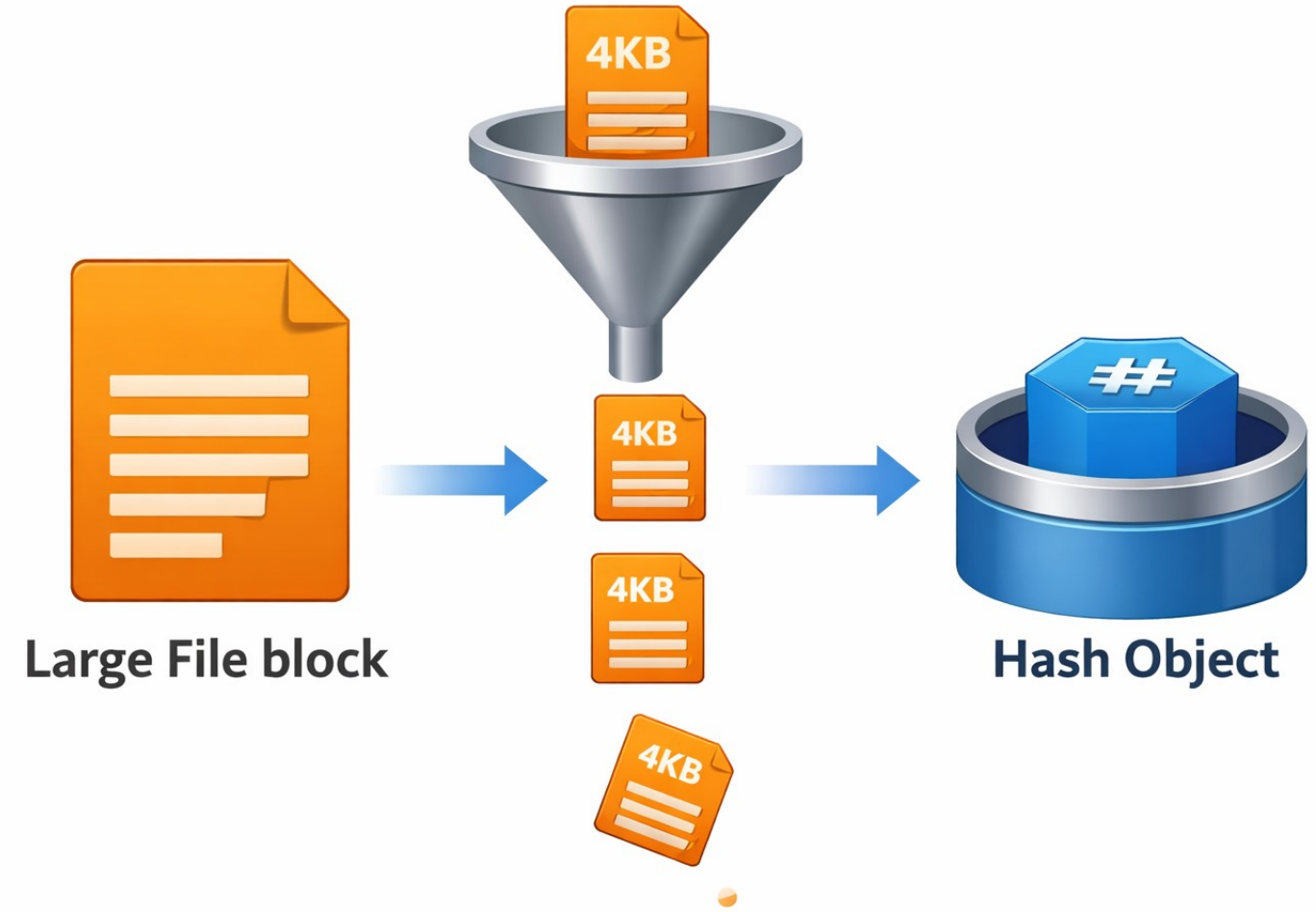
- SHA-256 is the current industry standard.
- Avoid MD5 or SHA-1 (cryptographically broken).

```
import hashlib
data = b"Hello World"
digest = hashlib.sha256(data).hexdigest()
```

Hashing Large Files



Hashing Large Files



Hashing Large Files

- Loading a 4GB file into RAM to hash it will crash your script.
- Solution:
  - Read and hash the file in small chunks using .update().

Why SHA-256 is Bad for Passwords

- Standard hashes are designed to be fast.
- Attackers with GPUs can guess billions of passwords per second.
- Vulnerable to "Rainbow Table" attacks (pre-computed hash lists).

Key Derivation & Salts

- Salts:
  - Random bytes added to a password before hashing. Defeats Rainbow Tables.
- KDFs (Key Derivation Functions): Intentionally slow down the hashing process (e.g., scrypt, pbkdf2).
- Defeats brute-force/GPU attacks.

Using hashlib.scrypt

- scrypt is CPU and Memory intensive.

```
salt = os.urandom(16)
hashed_pw = hashlib.scrypt(
    b"password123", salt=salt, n=16384, r=8,
p=1)
```

What is Shodan?

- Often called the "Search Engine for the Internet of Things".
- Google indexes web content; Shodan indexes internet-connected devices (servers, routers, webcams).
- Requires an API key and pip install shodan.

Content vs. Infrastructure

Google

Shodan

Content vs. Infrastructure

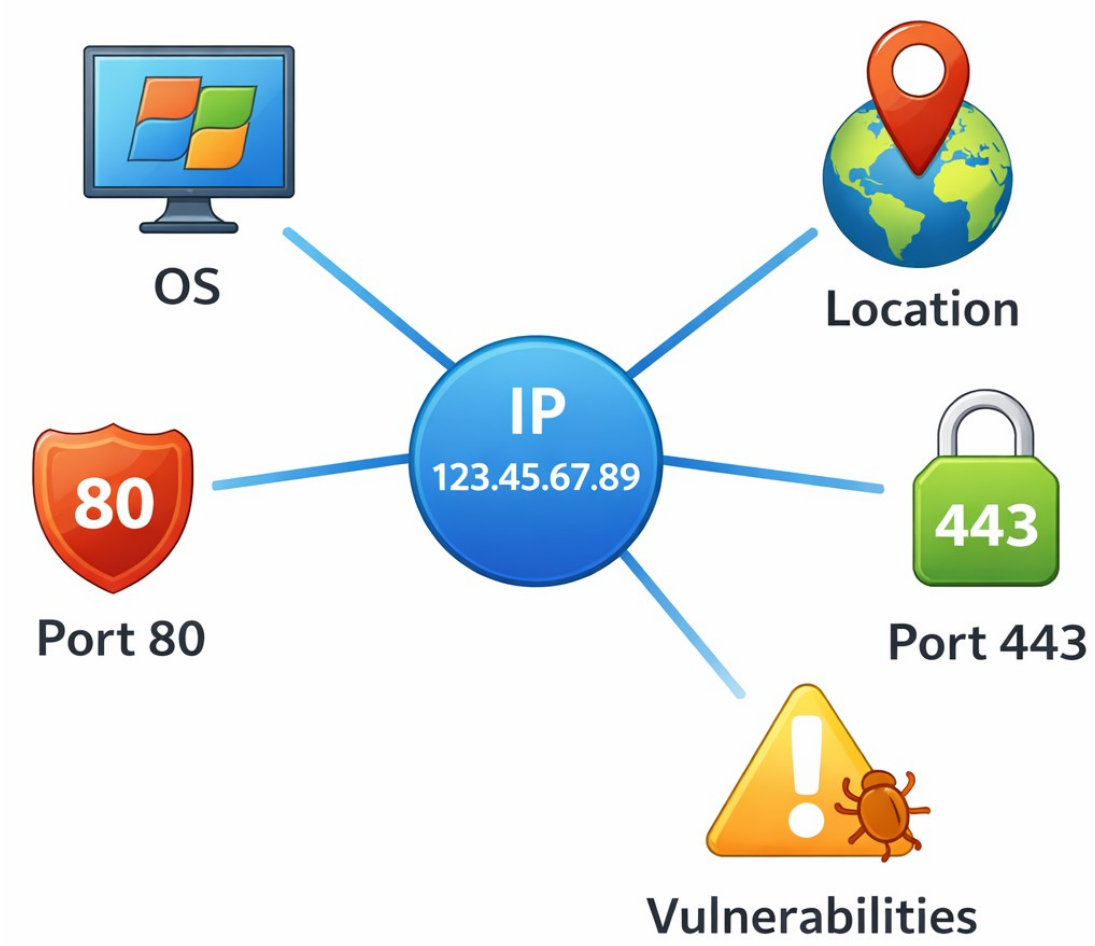
- Google Search:
  - "Find me websites talking about Apache servers."
- Shodan Search:
  - "Find me IP addresses running Apache server version 2.4.49."

Querying the Database

- Queries do not actively scan targets; they query Shodan's existing database.
- Returns IPs, Ports, Organizations, and Banners.

```
import shodan
api = shodan.Shodan("API_KEY")
results = api.search('apache')
```

Analyzing a Specific Target



Analyzing a Specific Target

- api.host(ip) returns everything Shodan knows about one IP address.
- Reveals operating systems, geographic location, open ports, and known vulnerabilities (CVEs).

Faceted Searching

- Used for macro-level statistics, not individual IPs.
  - E.g., "Show me the top 5 countries hosting exposed RDP servers."
  - Uses api.count() instead of api.search() to save API credits.

Network Alerts

- Shodan isn't just for attackers.
- Blue teams use it to monitor their own IP space.
- Set up alerts to trigger if Shodan detects a newly exposed port on your corporate network.

```
# Alert if any new port opens on this IP range
api.alerts.create("My Network", ["203.0.113.0/24"])
```

Key Takeaways

- datetime:
  - Always be aware of timezones (zoneinfo) to prevent logic bugs.
- hashlib:
  - Never use fast hashes (SHA-256, MD5) for passwords. Always use scrypt + Salts.
- shodan:
  - Powerful reconnaissance tool. Use it to map attack surfaces and monitor your own perimeter.

A Note on OSINT and Security

- Shodan data is public, but actively probing or exploiting systems without explicit, written permission is illegal.
- Always operate within ethical bounds and terms of service.

Thank you

@mitu_skillologies

@mITuSkillologies

@mitu_group

@mitu-skillologies

@MITUSkillologies

kaggle

@mituskillologies

Web Resources
https://mitu.co.in
http://tusharkute.com

@mituskillologies

contact@mitu.co.in

tushar@tusharkute.com

Public Feedback